T2 简单算法题（algorithm）题解

2026 赛季 CSP-S 复赛热身赛（03） · 时间 1.0 秒 / 内存 512 MiB · 答案保证在 64 位整数内

目 录

0 题意：这题要我们做什么
1 数据范围与解题思路
2 第一档：会判断，就能暴力数（8 分）
3 第二档：同高度的点两两配对（24 分）
4 第三档：不造串，按段算（48 分）
5 第四档：m1 = 1 的特殊结构（8 分）
6 正解：一次吞掉 need 个（100 分）
7 易错点清单
8 总结

0 题意：这题要我们做什么

一句话题意

给你 8 个整数，它们决定了一个很长的括号串 S 是怎么生成的。你要回答：**S 里有多少个非空子串是合法括号序列**。

整件事可以拆成三步，每一步都很机械：

1. 由输入**递推**出长度为 n 的序列 c；
2. 用 c **造出**括号串 S；
3. **数一数** S 里有多少个合法括号子串。

输入与输出

输入只有一行，8 个整数：

```
n   x   y   z   m0   m1   c0   c1
```

其中 c_0、c_1 由输入直接给出，从 i = 2 起按下面的式子递推（**注意 i 是从 0 开始数的**）：

$$c_i = (c_{i-1} \cdot x + c_{i-2} \cdot y + z) \bmod M_i + 1$$　（i ⩾ 2；i 为偶数时 M_i = m0，i 为奇数时 M_i = m1）

造串规则更简单——对 i = 0, 1, …, n−1：

造串 **i 为偶数，就在 S 末尾添 c_i 个 '('；i 为奇数，就添 c_i 个 ')'。**

输出一个整数：S 的非空子串中合法括号序列的个数（题目保证答案在 64 位有符号整数内，用 `long long`）。

什么是合法括号序列

题面用三条规则递归地定义，它其实等价于我们更熟悉的两条：

1. 左括号和右括号**数量相等**；
2. 从左往右看，任何时刻已经出现的 ')' 都**不超过**已经出现的 '('。

先自己判断几个，再往下读：

括号串	合法?	为什么
(|)	合法	1 对，中途 ')' 没超过 '('
((|))	合法	2 对，一对套着另一对
(|) | (|)	合法	两个 (|) 并排（题面的第 3 条）
) | (	不合法	第一个就是 ')'，一开始 ')' 就比 '(' 多
(|)) | (	不合法	数到第 3 个字符时已有 2 个 ')'、只有 1 个 '('
((|)	不合法	数量就不相等：2 个 '(' 对 1 个 ')'

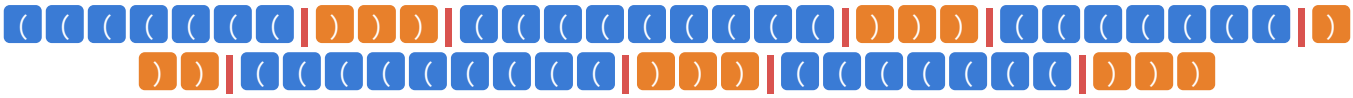
整道题的难点在第三步：S 可能长到根本造不出来，怎么数？这就是后面五档要解决的问题。

样例 1

输入 `10 686962104 149902149 5232363 10 4 7 3`，输出 **15**。这里 m0 = 10、m1 = 4、$c_0 = 7$、$c_1 = 3$。手算一步递推（i = 2 是偶数，所以 $M_2 = m0 = 10$）：

```
c2 = (c1 × x + c0 × y + z) mod m0 + 1
   = (3 × 686962104 + 7 × 149902149 + 5232363) mod 10 + 1
   = 3115433718 mod 10 + 1 = 8 + 1 = 9
```

全部算完得到 c = [7, 3, 9, 3, 7, 3, 9, 3, 7, 3]，造出的串如下（红色竖线是相邻两段的分界）：



串长 54，共 10 段：'(' 段与 ')' 段交替，每个 ')' 段都是 3 个 ')'

它是结构相同的 5 组，每组"若干个 '(' 接 3 个 ')'"。第 1 组里，3 个 ')' 从左往右各自配上它左边 1 个、2 个、3 个 '('，得到 (\|)、((\|))、(((\|))) 共 3 个。后 4 组一样，所以答案是 5 × 3 = **15**。

1 数据范围与解题思路

拿到一道题，先别急着写代码，先把范围看清楚——它能直接告诉你该往哪个方向想。

各个参数的范围

量	范围
n	$1 \leqslant n \leqslant 10^7$
x, y, z	$0 \leqslant x, y, z \leqslant 10^9$
c_0, m0	$1 \leqslant c_0 \leqslant m0 \leqslant 10^9$
c_1, m1	$1 \leqslant c_1 \leqslant m1 \leqslant 10^9$
答案、时限、内存	在 64 位整数内；1.0 秒；512 MiB

注意 c_i 的取值范围：$i \geqslant 2$ 时 $c_i \leqslant M_i$（取模再加 1），加上题面给出的 $c_0 \leqslant m0$、$c_1 \leqslant m1$，所以**每个 c_i 都不会超过对应的 M**。

子任务（subtask）

编号	分值	限制
#1	8	$n \leqslant 20$，$\max\{m0, m1\} \leqslant 500$
#2	16	$n \leqslant 5000$，$\max\{m0, m1\} \leqslant 4000$
#3	8	$n \leqslant 20$
#4	16	$n \leqslant 5000$
#5	8	$n \leqslant 10^6$，**m1 = 1**
#6	28	$n \leqslant 10^6$
#7	16	无特殊限制

本题是**捆绑测试**：同一个 subtask 里的测试点必须全对才得分，所以要吃下某一档，代码必须同时满足这一档的全部条件。

从这张表能读出什么

读法 做题前先问自己两个问题，答案就藏在上面两张表里。

1. **串造得出来吗？** 串长 = 所有 c_i 之和。#1 里串长 $\leqslant 20 \times 500 = 10^4$，#2 里 $\leqslant 5000 \times 4000 = 2 \times 10^7$，这两档**可以直接把串造出来**。可 #3 之后 m 没了上限，c_i 可达 10^9，串长可达 10^{16}——**造不出来，也不能在串上扫**，只能按"段"来算。
2. **复杂度能撑多大？** 时限 1 秒。$n \leqslant 20$ 时随便怎么写都行；$n \leqslant 5000$ 时 $O(n^2)$（约 2.5×10^7 次操作）勉强能过；n 到了 $10^6 \sim 10^7$，必须是 O(n)。

顺着这两个答案，一条升级路线就自然出来了，本文也按它组织：

档位	做法	解决的问题 / 还差什么
第 2 节	造串 + 前缀和判断 + 枚举	会数了，但 $O(\|S\|^2)$ 太慢 → 8 分
第 3 节	造串 + 单调栈	降到 $O(\|S\|)$，但仍要造串 → 24 分
第 4 节	不造串，按段算	串存得下了，但找匹配段要 $O(n^2)$ → 48 分
第 5 节	按段算 + 栈存整段（m1 = 1）	栈的骨架有了，但只适用于 m1 = 1 → 8 分
第 6 节	按段算 + 栈存整段（通用）	O(n)，全部数据 → 100 分

2 第一档：会判断，就能暴力数　8 分

适用 subtask #1（8 分）：$n \leqslant 20$，$\max\{m0, m1\} \leqslant 500$，串长不超过 10^4。

把括号变成数字：前缀和

要数合法子串，先得会判断"某一个子串合不合法"。逐个模拟太慢，我们把括号变成数字：**'(' 记作 +1，')' 记作 −1**。

然后求**前缀和**——就是"从开头一路加到当前位置的总和"。以 (|) | (|) 为例：

位置 i	0	1	2	3	4
字符	—	(	)	(	)
对应数值	—	+1	−1	+1	−1
P_i（加到 i 为止）	0	1	0	1	0

记 P_i 为前 i 个数的和，并规定 $P_0 = 0$（一个字符都还没看时，和为 0）。上表最后一行的 $P = [0, 1, 0, 1, 0]$ 就是它的前缀和。

用前缀和判断子串

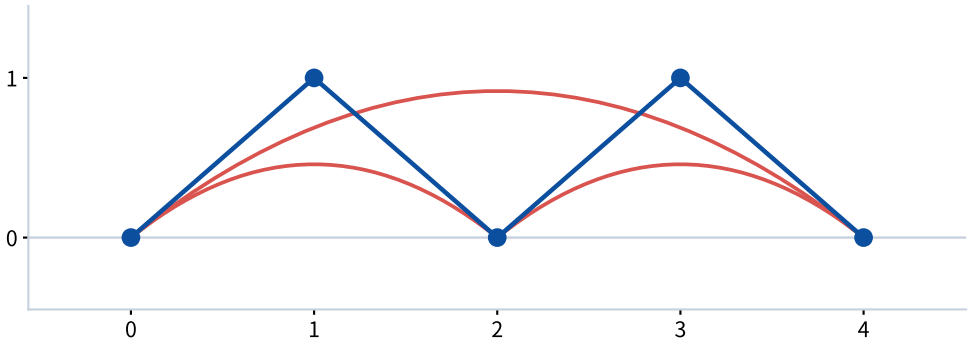
子串 S[j+1 .. r]（0 ⩽ j < r ⩽ |S|）合不合法，只看两件事：

> $P_r = P_j$ ① 配平：这一段里 '(' 和 ')' 一样多
>
> ② 不破底：$P_j, P_{j+1}, \ldots, P_r$ 中最小的一个 ⩾ P_j（这一段里 ')' 从没比 '(' 多）

两条同时成立 ⟺ 子串合法。用第 0 节那两个例子验一下：

- (|)：P = [0, 1, 0]，j = 0、r = 2，$P_2 = P_0 = 0$ 配平；最小值 min(0, 1, 0) = 0 ⩾ 0 没破底 → **合法**；
-) | (：P = [0, −1, 0]，j = 0、r = 2，$P_2 = P_0 = 0$ 也配平；但最小值 min(0, −1, 0) = −1 < 0 破底了 → **不合法**。

把 P 画成折线，每一条红色弧就对应一个合法子串：



(|) | (|)：3 条弧分别对应 S[1..2]、S[3..4]、S[1..4]，共 3 个

照着判据去枚举

造出串、算出 P，然后枚举左端 j，让 r 从 j+1 一直扫到 |S|，同时维护 mn = P_j 到 P_r 的最小值：

- mn < P_j → 已经破底，再往右只会更低，直接 break；
- P_r = P_j → 找到 1 个合法子串，答案加一。

手算 c = [3, 2] → S = (((|)) → P = [0, 1, 2, 3, 2, 1]

j	P_j	向右扫	找到
0	0	P 一路走到 1, 2, 3, 2, 1，再没回到 0	—
1	1	r = 5 时 $P_5 = 1$，途中最小值也是 1	((|))
2	2	r = 4 时 $P_4 = 2$，途中最小值也是 2	(|)
3	3	之后 P 只降到 1，回不到 3	—
4	2	之后 P 只降到 1，回不到 2	—

合计 **2** 个。

partial1_prefix.cpp · 造串 → 前缀和 → 枚举左端

```cpp
// T2 第一档：造串 + 前缀和 + 枚举起点，时间复杂度 O(|S|^2)
//
// 判据：记 P[0] = 0，'(' 记 +1、')' 记 -1。
//   子串 S[j+1 .. r] 是合法括号串 等价于 下面两条同时成立：
//     1. P[r] == P[j]          —— 左右配平
//     2. P[j..r] 的最小值 >= P[j]  —— 中间不会跌破起点（不会出现 )( 这种）
//
// 适用：Subtask #1，n <= 20 且 max{m0,m1} <= 500，串长 <= 10^4

#include <cstdio>

const int MAXL = 10000 + 5;      // 串的最大长度 20 * 500 = 10^4

char s[MAXL];                    // 造出来的括号串，下标 0 .. len-1
int  P[MAXL];                    // 前缀和，P[i] 是前 i 个字符的和

int main() {
    freopen("algorithm.in", "r", stdin);
    freopen("algorithm.out", "w", stdout);

    long long n, x, y, z, m0, m1, c0, c1;
    if (scanf("%lld %lld %lld %lld %lld %lld %lld %lld",
              &n, &x, &y, &z, &m0, &m1, &c0, &c1) != 8) return 0;

    // 1. 造串：c[0] = c0，c[1] = c1，i >= 2 时
    //    c[i] = (c[i-1]*x + c[i-2]*y + z) % M + 1，偶数 M = m0，奇数 M = m1
    //    i 为偶数加入 c[i] 个 '('，i 为奇数加入 c[i] 个 ')'
    int len = 0;
    long long pre1 = 0, pre2 = 0;            // pre1 = c[i-1]，pre2 = c[i-2]
    for (long long i = 0; i < n; i++) {
        long long c;
        if (i == 0) {
            c = c0;
        } else if (i == 1) {
            c = c1;
        } else {
            long long M = m0;                // i 为偶数用 m0，奇数用 m1
            if (i % 2 == 1) M = m1;
            c = (pre1 * x + pre2 * y + z) % M + 1;
```

```
        }
        pre2 = pre1;
        pre1 = c;

        char ch = '(';
        if (i % 2 == 1) ch = ')';
        for (long long k = 0; k < c; k++) s[len++] = ch;
    }

    // 2. 算前缀和：P[0] = 0，遇 '(' 加一，遇 ')' 减一
    for (int i = 0; i < len; i++) {
        if (s[i] == '(') P[i + 1] = P[i] + 1;
        else             P[i + 1] = P[i] - 1;
    }

    // 3. 枚举起点 j，向右扩展右端点 r，同时维护 P[j..r] 的最小值 mn
    long long ans = 0;
    for (int j = 0; j < len; j++) {
        int mn = P[j];
        for (int r = j + 1; r <= len; r++) {
            if (P[r] < mn) mn = P[r];
            if (mn < P[j]) break;            // 跌破起点，后面不可能再合法
            if (P[r] == P[j]) ans++;         // 配平，找到一个合法子串
        }
    }

    printf("%lld\n", ans);
    return 0;
}
```

不够 复杂度 O(|S|^2)，串长上万就跑不动了。下一节把它降到 O(|S|)。

3 第二档：同高度的点两两配对 24 分

适用 subtask #1、#2（合计 24 分）：串长不超过 5000 × 4000 = 2 × 10^7。

换个角度看判据

上一档是在"一个一个找子串"：定住起点，往右找终点。反过来想：两个下标 j < r，什么时候能组成一个合法子串？按判据，就是 $P_j = P_r$ 且它们之间没有更低的点。也就是说：

观察 **两个高度相同的点，只要中间没有被更低的点隔开，就能配成一个合法子串。**

按这条把 P_0 到 $P_{|S|}$ 分成若干**层**：同一层的点高度相同，且中间没有比它们更低的点。高度不同的点配不上（不满足配平），同高度却被更低点隔开的也配不上（破底），所以合法子串只在层内产生。一层有 c 个点，任取两个都能配上，于是：

> 一层有 c 个点 → 贡献 $\frac{c(c-1)}{2}$ 个合法子串 答案 = 每一层的贡献之和

（这就是组合里的 $C(c,2)$ ：c 个点里选 2 个，不讲顺序。）

用栈把每一层找出来

栈就是"只有顶部开口的盒子"，只能从顶上放进去、取出来。我们从左到右扫 P_0 到 $P_{|S|}$，让栈保存"已经开始、还没被截断的层"，每层记两个数：高度、已有点数，且高度从栈底到栈顶严格递增。扫到值 v 时：

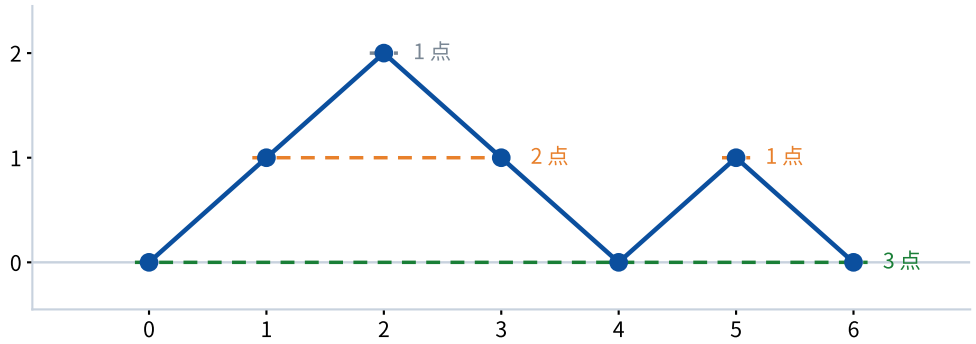
- **先弹**：栈顶那些高度 > v 的层，被这个更低的点截断了，再也不可能有新点进来，逐个弹出并结算 $c(c-1)/2$ ；
- **再并**：弹完后若栈顶高度 = v，说明 v 属于这一层，点数加一；
- **否则压**：栈已空，或栈顶高度 < v，新开一层 (v, 1) 压栈。

扫完以后，栈里剩下的层也要逐个弹出结算。每个点进栈一次、出栈一次，所以总复杂度是 O(|S|)。

手算 S = ((|)) | (|) → P = [0, 1, 2, 1, 0, 1, 0]

i	$v = P_i$	动作	栈（底 → 顶）	结算
0	0	压入新层	(0, 1)	—
1	1	压入新层	(0, 1)　(1, 1)	—
2	2	压入新层	(0, 1)　(1, 1)　(2, 1)	—
3	1	弹 (2, 1)，并入高度 1	(0, 1)　(1, 2)	0
4	0	弹 (1, 2)，并入高度 0	(0, 2)	1
5	1	压入新层	(0, 2)　(1, 1)	—
6	0	弹 (1, 1)，并入高度 0	(0, 3)	0
扫完收尾		弹出 (0, 3)	空	3

合计 0 + 1 + 0 + 3 = **4** 个，正好是 ((|)) 、(|) 、(|) 、((|)) | (|) 。



分层结果：高度 0 的 3 个点（i = 0, 4, 6）同层，配 3 对；高度 1 的点被 i = 4 的 0 隔开，分成 {1, 3} 和 {5} 两层，前者配 1 对；高度 2 只有 1 个点。共 3 + 1 = 4

partial2_monostack.cpp · 造串 → 前缀和 → 单调栈分层

```
// T2 第二档：造串 + 前缀和 + 单调栈分层，时间复杂度 O(|S|)
//
// 判据和第一档一样：两点前缀和相等、且中间不低于它，就能配成一个合法子串。
// 把前缀和看成地形，同一层的点只要没被更低的点截断，层内任意两点都能配对。
// 用一个值递增的栈维护还没被截断的层：
//   遇到更低的值 → 比它高的层被截断，弹出并结算 C(cnt, 2);
//   遇到相同的值 → 并入这一层，点数加一;
```

```cpp
//   遇到更高的值 → 新开一层压栈。
//
// 适用：Subtask #1、#2，串长 <= 2*10^7

#include <cstdio>

const int MAXL = 20000000 + 5;   // 串的最大长度 5000 * 4000 = 2*10^7

char s[MAXL];                    // 造出来的括号串
int  P[MAXL];                    // 前缀和
int  stVal[MAXL];                // 栈：这一层的前缀和的值，从底到顶严格递增
int  stCnt[MAXL];                // 栈：这一层已经放了几个点

int main() {
    freopen("algorithm.in", "r", stdin);
    freopen("algorithm.out", "w", stdout);

    long long n, x, y, z, m0, m1, c0, c1;
    if (scanf("%lld %lld %lld %lld %lld %lld %lld %lld",
              &n, &x, &y, &z, &m0, &m1, &c0, &c1) != 8) return 0;

    // 1. 造串（同第一档）
    int len = 0;
    long long pre1 = 0, pre2 = 0;            // pre1 = c[i-1], pre2 = c[i-2]
    for (long long i = 0; i < n; i++) {
        long long c;
        if (i == 0) {
            c = c0;
        } else if (i == 1) {
            c = c1;
        } else {
            long long M = m0;
            if (i % 2 == 1) M = m1;
            c = (pre1 * x + pre2 * y + z) % M + 1;
        }
        pre2 = pre1;
        pre1 = c;

        char ch = '(';
        if (i % 2 == 1) ch = ')';
        for (long long k = 0; k < c; k++) s[len++] = ch;
    }

    // 2. 算前缀和（同第一档）
    for (int i = 0; i < len; i++) {
        if (s[i] == '(') P[i + 1] = P[i] + 1;
        else             P[i + 1] = P[i] - 1;
    }

    // 3. 单调栈分层，扫一遍 P[0..len]
    long long ans = 0;
    int top = 0;                             // 栈顶下标，栈空时 top == 0
    for (int i = 0; i <= len; i++) {
        int v = P[i];
        int cnt = 0;
        while (top > 0 && stVal[top - 1] > v) {      // 更高的层被截断，结算
            long long t = stCnt[top - 1];
```

```
            ans = ans + t * (t - 1) / 2;              // 层内两两配对，C(t, 2)
            top--;
        }
        if (top > 0 && stVal[top - 1] == v) {         // 同高度，并入这一层
            cnt = stCnt[top - 1];
            top--;
        }
        stVal[top] = v;                    // 压入新层（或合并后的层）
        stCnt[top] = cnt + 1;
        top++;
    }
    while (top > 0) {                      // 收尾：栈里剩下的层也要结算
        long long t = stCnt[top - 1];
        ans = ans + t * (t - 1) / 2;
        top--;
    }

    printf("%lld\n", ans);
    return 0;
}
```

不够　它仍然要造串。m 没有上限时 c_i 可达 10^9，串长可达 10^{16}，根本存不下。下一节改成"按段算"。

4 第三档：不造串，按段算　48 分

适用 subtask #1 ~ #4（合计 48 分）：n ⩽ 5000。

上一档已经能在串上用 O(|S|) 数完。可 m 没有上限时 c_i 可达 10^9，串长可达 10^{16}——串根本造不出来。这一档改成**只在"段"上算**。

段：连续相同的一块

把连续的 c_i 个相同括号看成**一段**：段号 i 从 0 起，**偶数段全是 '('，奇数段全是 ')'**，整个串就是这 n 段顺次拼起来。段只有 n 个，可每一段里可能有上亿个括号，所以只能**整段整段地处理**，绝不能把段拆成一个一个字符来看。



c = [2, 3, 2] 造出的串，共 7 个字符、3 段（红色竖线是段的分界）

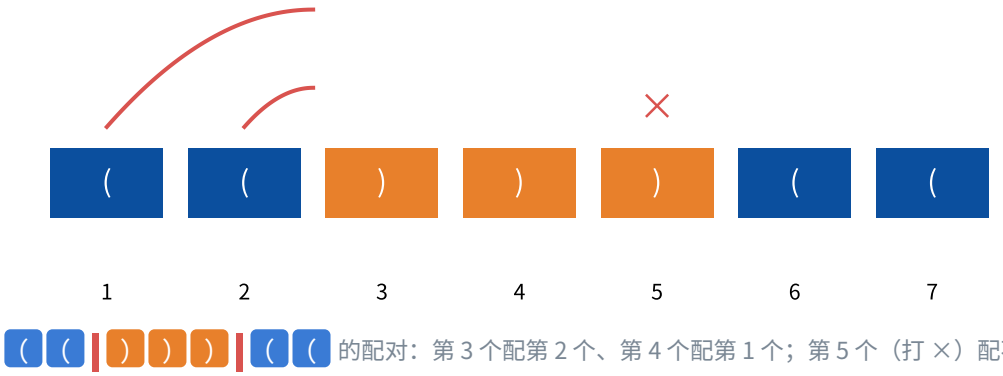
先想清楚：怎么数才不会漏

合法括号串一定是以 ')' 结尾的，所以可以按"结尾"来数：对每个位置算出"以它结尾的合法子串有几个"，全部加起来就是答案。为此定义（**位置从 1 开始数**，和题面的 [l, r] 一致）：

> **dp[r]** = 以第 **r** 个字符结尾的合法子串个数
>
> 第 r 位是 '(' 时 dp[r] = 0——以 '(' 结尾的串不可能合法。约定 dp[0] = 0。

dp 怎么算：靠"配对"

一个括号串里谁跟谁配对是**唯一确定**的：从左往右看，每个 ')' 都和它左边最近的、还没被配走的 '(' 配成一对；轮到某个 ')' 时左边已经没有空闲的 '(' 了，它就是多出来的。



的配对：第 3 个配第 2 个、第 4 个配第 1 个；第 5 个（打 ×）配不上

设第 r 位的 ')' 配到了第 l 位的 '('，那么：

$$dp[r] = 1 + dp[l-1]$$

l = 1 时 l−1 = 0，按约定 dp[0] = 0。

这个式子由三件事组成：

- **那个 1**：l 和 r 既然配成一对，S[l..r] 自己就是一个合法子串（外面一对，里面还包着一个合法串）；
- **那个 dp[l−1]**：紧贴在 l 左边、以位置 l−1 结尾的每一个合法子串，接上 S[l..r] 之后仍然合法——"两个合法串并排"还是合法串；
- **再没有别的可能**：以 r 结尾的合法子串，最外面一层必然就是 l 与 r 这一对，所以它只能是"（以 l−1 结尾的合法串，或者干脆没有）+ 这一对"。

手算 S = ( ) ( )

位置 r	1	2	3	4
字符	(	)	(	)
配到 l	—	1	—	3
dp[r]	0	$1 + dp[0] = 1$	0	$1 + dp[2] = 2$

答案 = dp[2] + dp[4] = 1 + 2 = **3**，正好是 ( )、( )、( ) ( )。第 4 位那个 dp = 2，说的就是"以它结尾的合法子串有 2 个"：( ) 和 ( ) ( )。

把 dp 搬到段上

段里的括号太多了，不能一个一个算。好在绝大多数 ')' 的贡献都一样——只要看它配到的那个 l 是不是**段首**（所在 '(' 段最左边的那个 '('）：

- **l 不是段首** → l−1 还是本段的 '('，dp[l−1] = 0，贡献就是 **1**；
- **l 是段首** → l−1 是上一段的最后一个位置，贡献 $1 + E[s-1]$，其中 s 是这个 '(' 段的段号。

为什么"段首最后才被配掉"？因为每个 ')' 都配最近的未匹配 '('，所以一个 '(' 段是**从右往左**被消耗的，最左边的段首当然最后才轮到。

这里的 $E[i]$ 定义为：**')' 段 i 末尾那个位置的 dp 值**（只对 ')' 段有意义）。'(' 段 s 的前一段 s−1 一定是 ')' 段，所以"上一段末尾"就是 $E[s-1]$；s = 0 时前面没有段，按 0 算。

手算 c = [2, 3, 2] → ((\|))) \| ((

处理到	配到的 '('	贡献	处理之后
段 1 第 1 个 ')'	段 0 的第 2 个（不是段首）	1	段 0 还剩 1 个
段 1 第 2 个 ')'	段 0 的第 1 个（**段首**）	$1+0=1$	段 0 用完（s = 0，前面没有段）
段 1 第 3 个 ')'	没有 '(' 可配	0	$E[1]=0$
段 2（'(' 段）	整段先留着，等后面的 ')' 来配	—	还剩 2 个

答案 = 1 + 1 = **2**，也就是 (\|) 和 ((\|))。

于是整题只要维护两个数组：$rem[s]$ 记 '(' 段 s 还剩几个 '('，$E[i]$ 记 ')' 段 i 末尾的 dp 值。处理一个 ')' 段时，从紧邻左边的 '(' 段开始往前找，剩余为 0 就跳过，直到本段的 ')' 全配完。

partial3_seg.cpp · 按段处理，匹配段用朴素查找

```cpp
// T2 第三档：不再造串，直接按"段"计算；找匹配段时从后往前逐个扫，最坏 O(n^2)
//
// 段：第 i 段是连续 c[i] 个相同括号，i 为偶数全是 '('，i 为奇数全是 ')'。
// rem[s] = '(' 段 s 还剩几个左括号没被匹配
// E[i]   = ')' 段 i 最后一个位置的 dp 值，即以该位置结尾的合法子串数
//
// 适用：Subtask #1 ~ #4，n <= 5000，O(n^2) 约 2.5*10^7 完全够用
// 数组按 n 的上界 10^7 开，更大的 n 也能跑，只是会非常慢

#include <cstdio>

const int MAXN = 10000000 + 5;   // n 最大 10^7

long long c[MAXN];           // c[i] = 第 i 段有多少个括号
long long rem[MAXN];         // rem[s] = '(' 段 s 剩余的 '(' 个数
long long E[MAXN];           // E[i] = ')' 段 i 末尾位置的 dp 值

int main() {
    freopen("algorithm.in", "r", stdin);
    freopen("algorithm.out", "w", stdout);

    long long n, x, y, z, m0, m1, c0, c1;
    if (scanf("%lld %lld %lld %lld %lld %lld %lld %lld",
              &n, &x, &y, &z, &m0, &m1, &c0, &c1) != 8) return 0;

    // 1. 递推算出 c 数组
    for (long long i = 0; i < n; i++) {
        if (i == 0) {
            c[i] = c0;
        } else if (i == 1) {
            c[i] = c1;
        } else {
            long long M = m0;            // i 为偶数用 m0，奇数用 m1
            if (i % 2 == 1) M = m1;
            c[i] = (c[i - 1] * x + c[i - 2] * y + z) % M + 1;
        }
```

```
    }

    // 2. 初始化：每个 '(' 段一开始有 c[i] 个左括号等着被匹配
    for (long long i = 0; i < n; i = i + 2) rem[i] = c[i];

    long long ans = 0;
    for (long long i = 1; i < n; i = i + 2) {        // 处理每个 ')' 段
        long long need = c[i];                       // 还要配掉多少个 '('
        long long last = 0;                          // 本段末尾位置的 dp 值
        long long s = i - 1;                         // 从紧邻的 '(' 段往前找
        while (need > 0 && s >= 0) {
            if (rem[s] == 0) {                       // 这段已经用光，继续往前
                s = s - 2;
                continue;
            }
            long long d;                             // 这一段能消化掉 d 个
            if (rem[s] < need) d = rem[s];
            else               d = need;
            rem[s] = rem[s] - d;
            need = need - d;
            if (rem[s] == 0) {
                // 整段被消耗完：前 d-1 个位置各贡献 1
                ans = ans + (d - 1);
                // 最后一个位置是"段首"，它前面还能接上一段，贡献 1 + E[s-1]
                long long val = 1;
                if (s >= 1) val = val + E[s - 1];
                ans = ans + val;
                last = val;
                s = s - 2;
            } else {
                // 只消耗了一部分：这些位置每个都只贡献 1
                ans = ans + d;
                last = 1;
            }
        }
        if (need > 0) last = 0;                  // 还有 ')' 没配上，末尾 dp = 0
        E[i] = last;
    }

    printf("%lld\n", ans);
    return 0;
}
```

不够 每个 ')' 段最坏要往前扫过 O(n) 段，总复杂度 $O(n^2)$。下一节用栈把"找匹配段"这一步省掉。

5 第四档：m1 = 1 的特殊结构 8 分

第三档慢在"每个 ')' 段都要从后往前重新找没用完的 '(' 段"。改法很自然：把还没用完的 '(' 段**整段压进栈**，用完就弹出，这样每段最多被弹一次。先在结构最简单的 subtask #5（8 分）上把这个骨架写出来。

m1 = 1 时，每个 ')' 段只有 1 个 ')'

题面保证 $1 \leqslant c_1 \leqslant m1$，所以 m1 = 1 时 $c_1 = 1$；而 $i \geqslant 2$ 的奇数段都有 $c_i = (\cdots) \bmod 1 + 1 = 1$ 。**每个 ')' 段恰好 1 个 ')'**，每次只要给这唯一的 ')' 配 1 个 '('。

于是每个 ')' 段只看栈顶

栈里存的是"还没用完的 '(' 段"，每个元素记两样：段号 s、还剩几个 '('。栈顶就是要被消耗的那一段。按第三档的结论，只有两种情况：

- **栈空** → 配不上，本段 $E[i] = 0$ ；
- **栈顶剩余 > 1** → 用掉一个，配到的是段中间的 '('（不是段首），贡献 **1**，本段 $E[i] = 1$ ；
- **栈顶剩余 = 1** → 配到的正是段首，贡献 $1 + E[s-1]$ ，本段 $E[i]$ 也等于它，然后这一段用完、弹栈。

每段最多进栈一次、弹出一次 → O(n)。

partial4_sub5.cpp · subtask #5 特化：栈存 '(' 段，每次配 1 个

```cpp
// T2 第四档：subtask #5 特化 —— m1 = 1 时，每个 ')' 段只有 1 个右括号
//
// 关键性质：m1 = 1 时 c1 <= m1 = 1，而 i >= 2 的奇数段
//           c[i] = (c[i-1]*x + c[i-2]*y + z) mod 1 + 1 = 1
//           所以每个 ')' 段恰好 1 个 ')'，只要给它配 1 个 '(' 就完事。
// 做法：栈里存还没用完的 '(' 段（段号 + 剩余个数），每个 ')' 段只看栈顶：
//   栈顶剩余 > 1  → 减一即可，贡献 1
//   栈顶剩余 == 1 → 这一整段用完，弹栈，贡献 1 + E[段号-1]（它是段首）
//
// 注意：这份代码只在 m1 = 1 时正确，见题解第四档。

#include <cstdio>

const int MAXN = 10000000 + 5;   // 按 n 的上界 10^7 开（m1 = 1 的数据也可能很大）

long long E[MAXN];           // E[i]: ')' 段 i 末尾位置的 dp 值
int      seg[MAXN];          // 栈：段号
long long lef[MAXN];         // 栈：该段剩余的 '(' 个数

int main() {
    freopen("algorithm.in", "r", stdin);
    freopen("algorithm.out", "w", stdout);

    long long n, x, y, z, m0, m1, c0, c1;
    if (scanf("%lld %lld %lld %lld %lld %lld %lld %lld",
              &n, &x, &y, &z, &m0, &m1, &c0, &c1) != 8) return 0;

    long long ans = 0;
    int top = 0;                              // 栈顶下标，栈空时 top == 0
    long long pre1 = 0, pre2 = 0;             // c[i-1]、c[i-2]，滚动递推

    for (long long i = 0; i < n; i++) {
        long long c;
        if (i == 0) {
            c = c0;
        } else if (i == 1) {
            c = c1;
```

```
        } else {
            long long M = m0;                   // i 为偶数用 m0，奇数用 m1
            if (i % 2 == 1) M = m1;
            c = (pre1 * x + pre2 * y + z) % M + 1;
        }
        pre2 = pre1;
        pre1 = c;

        if (i % 2 == 0) {
            seg[top] = (int)i;                  // '(' 段：整段压栈
            lef[top] = c;
            top++;
        } else {
            // m1 = 1，这里的 c 恒为 1，只需要配掉一个 '('
            if (top == 0) {
                E[i] = 0;                       // 前面没有 '(' 可配
            } else if (lef[top - 1] > 1) {
                lef[top - 1] = lef[top - 1] - 1;
                ans = ans + 1;                  // 左边还有同段的 '('，贡献 1
                E[i] = 1;
            } else {
                // 栈顶这段正好用完，最后这个是"段首"，左边是上一段
                long long val = 1;
                if (seg[top - 1] >= 1) val = val + E[seg[top - 1] - 1];
                ans = ans + val;
                E[i] = val;
                top--;                          // 整段用完，弹栈
            }
        }
    }

    printf("%lld\n", ans);
    return 0;
}
```

局限 它只在 m1 = 1 时正确：m1 ≠ 1 时 ')' 段有多个 ')'，每次只配 1 个会漏算。但"栈存整段、用完即弹"的骨架已经成型，下一节只差一步。

6 正解：一次吞掉 need 个 100 分

适用 全部 subtask（100 分）：n 可达 10^7。

只差一件事：')' 段可能有多个 ')'，不能每次只配 1 个。处理 ')' 段 i 时，令 need = c_i（还差几个 '(' 要配），然后反复从栈顶取：

- $d = \min(\text{lef},\ \text{need})$ ：这一轮从栈顶这段取走 d 个（lef 是栈顶剩余，need 是还差几个）；
- **d < lef**（这段只用掉一部分）：这 d 个 ')' 配到的都是段中间的 '('，各贡献 **1**，然后把 lef 减 d；
- **d = lef**（这段正好用完）：前 d−1 个各贡献 **1**，最后那一个配到的是段首，贡献 $1 + E[s-1]$ ，然后把这段弹出；
- **need 还没配完、栈却空了** → 多出来的那些 ')' 配不上，本段 $E[i] = 0$ 。

每段最多进栈一次、弹出一次 → **O(n)**。

full_solution.cpp · 栈存整段，一次吞 need 个，O(n)

```cpp
// T2 正解：按段处理 + 栈里存"整段"，时间复杂度 O(n)
//
// 和第三档一样按段算，区别只在"怎么找匹配段"：
// 第三档每个 ')' 段都要从后往前重新扫一遍，最坏 O(n^2)；
// 把没用完的 '(' 段整段压栈，用完就弹出，每段最多弹一次 → O(n)
//
// 栈的两个数组：seg[top] = 段号，lef[top] = 该段还剩几个 '('
// E[i] = ')' 段 i 末尾位置的 dp 值，即"以该位置结尾的合法子串数"

#include <cstdio>

const int MAXN = 10000000 + 5;

long long E[MAXN];           // E[i]: ')' 段 i 末尾位置的 dp 值
int      seg[MAXN];          // 栈：段号
long long lef[MAXN];         // 栈：该段剩余的 '(' 个数

int main() {
    freopen("algorithm.in", "r", stdin);
    freopen("algorithm.out", "w", stdout);

    long long n, x, y, z, m0, m1, c0, c1;
    if (scanf("%lld %lld %lld %lld %lld %lld %lld %lld",
              &n, &x, &y, &z, &m0, &m1, &c0, &c1) != 8) return 0;

    long long ans = 0;
    int top = 0;                             // 栈顶下标，栈空时 top == 0
    long long pre1 = 0, pre2 = 0;            // c[i-1]、c[i-2]，滚动递推

    for (long long i = 0; i < n; i++) {
        long long c;
        if (i == 0) {
            c = c0;
        } else if (i == 1) {
            c = c1;
        } else {
            long long M = m0;                // i 为偶数用 m0，奇数用 m1
            if (i % 2 == 1) M = m1;
            c = (pre1 * x + pre2 * y + z) % M + 1;
        }
        pre2 = pre1;
        pre1 = c;

        if (i % 2 == 0) {
            // '(' 段：整段压栈，等着后面的 ')' 来配
            seg[top] = (int)i;
            lef[top] = c;
            top++;
        } else {
            // ')' 段：从栈顶开始配，配完一个段就弹一个段
            long long need = c;              // 还要配掉多少个 '('
            long long last = 0;              // 本段末尾位置的 dp 值
            while (need > 0 && top > 0) {
                long long d;                 // 栈顶这一段能消化掉 d 个
                if (lef[top - 1] < need) d = lef[top - 1];
```

```
                else                        d = need;
                if (d == lef[top - 1]) {
                    // 整段用完：前 d-1 个位置各贡献 1
                    ans = ans + (d - 1);
                    // 最后一个是段首，左边是上一段，贡献 1 + E[段号-1]
                    long long val = 1;
                    if (seg[top - 1] >= 1) val = val + E[seg[top - 1] - 1];
                    ans = ans + val;
                    last = val;
                    top--;                      // 这个段用完了，弹出
                } else {
                    // 只用了一部分：这些位置每个贡献 1
                    ans = ans + d;
                    last = 1;
                    lef[top - 1] = lef[top - 1] - d;
                }
                need = need - d;
            }
            if (need > 0) last = 0;       // 栈空了还有 ')' 剩下，末尾 dp = 0
            E[i] = last;
        }
    }

    printf("%lld\n", ans);
    return 0;
}
```

要点 用 `pre1 / pre2` 滚动递推 c，不必开数组；偶数段整段压栈，奇数段用 `need` 循环吞栈，`d == lef[top-1]` 就说明这一整段用完了（最后配掉的是段首）。

7 易错点清单

1. **M_i 要看 i 的奇偶**：偶数取 m0、奇数取 m1；c_0、c_1 由输入直接给出，**只有 i ≥ 2 才递推**。
2. **递推的乘法要用 long long**：$c_{i-1} \cdot x$ 可达 10^{18}，int 会溢出。
3. **答案用 long long 输出**：题目只保证在 64 位内，超过 int 很常见。
4. **判合法的两条要同时判**：只判 $P_r = P_j$ 会把 `)` | `(` 也算成合法——它配平，但破了底。
5. **单调栈要收尾**：扫描结束后，栈里剩下的层别忘了再结算一次。
6. **段首的贡献是 $1 + E[s-1]$ 而不是 1**：这是最核心的坑。漏掉它，`(` | `)` | `(` | `)` 的第 4 位只会算出 1（正确答案 2），总数从 3 变成 2。
7. **本段有 ')' 没配上时，$E[i]$ 记 0**：末尾那个 ')' 没有对应的 '('，以它结尾的合法子串数是 0。
8. **消耗顺序是段尾 → 段首**：整段用完时，是"前 d−1 个各贡献 1 + 最后 1 个贡献 $1 + E[s-1]$ "，别把 d 个全当成 1，也别全当成 $1 + E[s-1]$ 。

8 总结

总结 **判一个子串合不合法**，靠前缀和的两条：配平、不破底。**数出所有合法子串**有两条路：串短就在串上做——把"同高度、没被隔开的点"分成层，层内两两配对；串太长就按段做——按"每个 ')' 结尾"记 dp，$dp[r]=1+dp[l-1]$ ，再配一个存整段的栈，其中段首的贡献 $1+E[s-1]$ 是关键。

四档的递进：枚举太慢 → 单调栈提速，但仍要造串 → 改按段算，可找匹配段太慢 → 在 m1 = 1 上做出"栈存整段"，再推广到任意 c_i。